



JAVA

- Java is one of the fundamental languages that produces software for multiple platforms.

PYTHON

- Python is readable, efficient and powerful high level programming language.

JAVA

- Statically typed and faster than python.
- Longer lines of code while compare to python.
- Popular for mobile and web applications.
- Java is a compiled programming language and the source code is compiled down to bytecode by the java compiler.

PYTHON

- Dynamically typed and slower than java.
- Shorter lines of code when compared to java.
- Popular for data science, ML, AI and IoT.
- Python is an interpreted language, the translation occurs at the same time as the program is being executed.

JAVA

- Java is an object oriented programming language.
- Supports encapsulation, inheritance, polymorphism and abstraction.

PYTHON

- Python is also object oriented language and it also a scripting language and it is easy to write script.

C++

- C++ is platform – dependent
- C++ is mainly used for system programming.
- C++ supports goto statement, multiple inheritance, operator overloading etc.
- It also supports pointers externally. We can access a particular memory location by using pointers.

JAVA

- Java is platform independent.
- Java is mainly used for application programming used in windows, web – based mobile applications.
- Java doesn't support multiple inheritance through class.
- Java supports pointers internally. To provide simplicity to developer, java restricted use of pointers.

C++

- C++ is a compiled language.
- C++ program is to be compiled by a standard c++ compiler.
- C++ supports both call by value and call by reference.

JAVA

- Java is compiled and interpreted both.
- Java source code is converted into bytecode at compilation time. The interpreter executes this bytecode at run time and produces output.
- Java supports call by value only. There is no call by reference in java

C++

- C++ doesn't have built – in support for threads. It relies on third – party libraries for thread support.
- C++ supports virtual keyword so that we can decide whether or not override a function.

JAVA

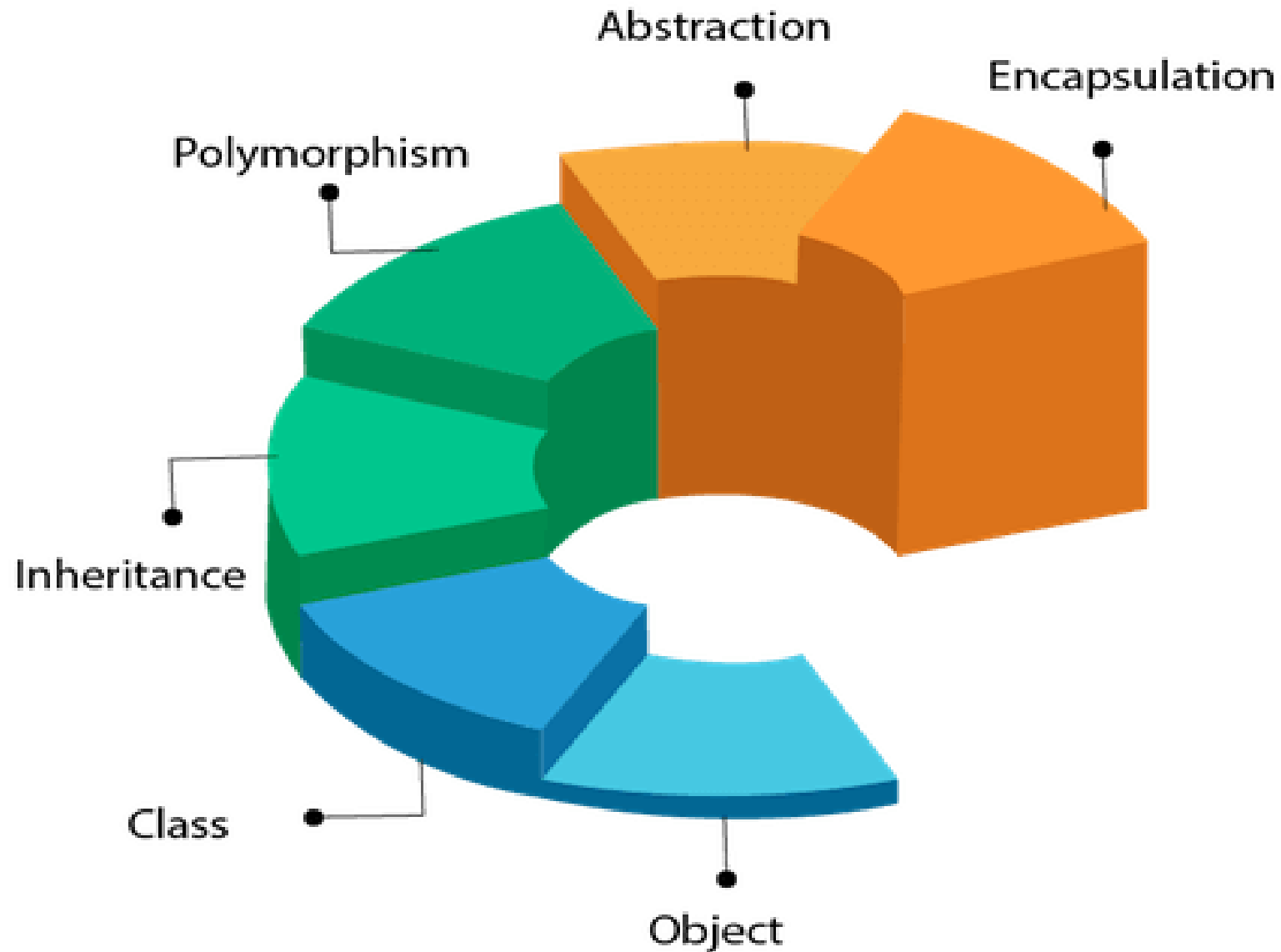
- Java has built – in thread support.
- Java doesn't support header files like C++. Java uses import keyword to class and methods.
- Java has no virtual keyword. We can override all non static keyword by default.

	C	C++	JAVA	PYTHON
Language Type	Procedure Oriented Language	Object Oriented Language	Object Oriented Language	Both Procedural & Object Oriented Language
Developer	Dennis Ritchie	Bjarne Stroustup	James Gosling	Guido Van Rossum
Building Block	Function Driven	Object Driven	Both Object and Class Driven	Function, Object and Class Driven
Extension	.c	.cpp	.java	.py
Platform	Dependent	Independent	Independent	Independent
Comment Style	/* */	// Single Line /* */ Multi-Line	// Single Line /* */ Multi-Line	# Single Line "" "" Multi Line
Keywords	32	50	63	33
Dynamic Variable	not supported	not supported	not supported	support
Data Security	Not Secure	Secure (less than Java)	Fully Secured	Secure (less than Java)
Case Sensitive	Yes	Yes	Yes	Yes
Header Files	Supported	Supported	Import Pacackages	Import Packages
Pointers	Supported	Supported	Not Supported	Not Supported
Exception Handling	No	Yes	Yes	Yes
Translation Type	Compiled	Compiled	Compiled & Interpreted	Interpreted

Features of Object-Oriented Programming (OOP)

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

OOPs (Object-Oriented Programming System)



Object

- An object is an entity that possesses both state and behavior.
- state means data and behavior means functionality.
- Object is a runtime entity, it is created at runtime.
- Object is an instance of a class. All the members of the class can be accessed through object.
- Examples of objects include a chair, pen, table, keyboard, and bike. Objects can be either physical or logical in nature.

Characteristics of Object

A

State

Represents the data of an object.

B

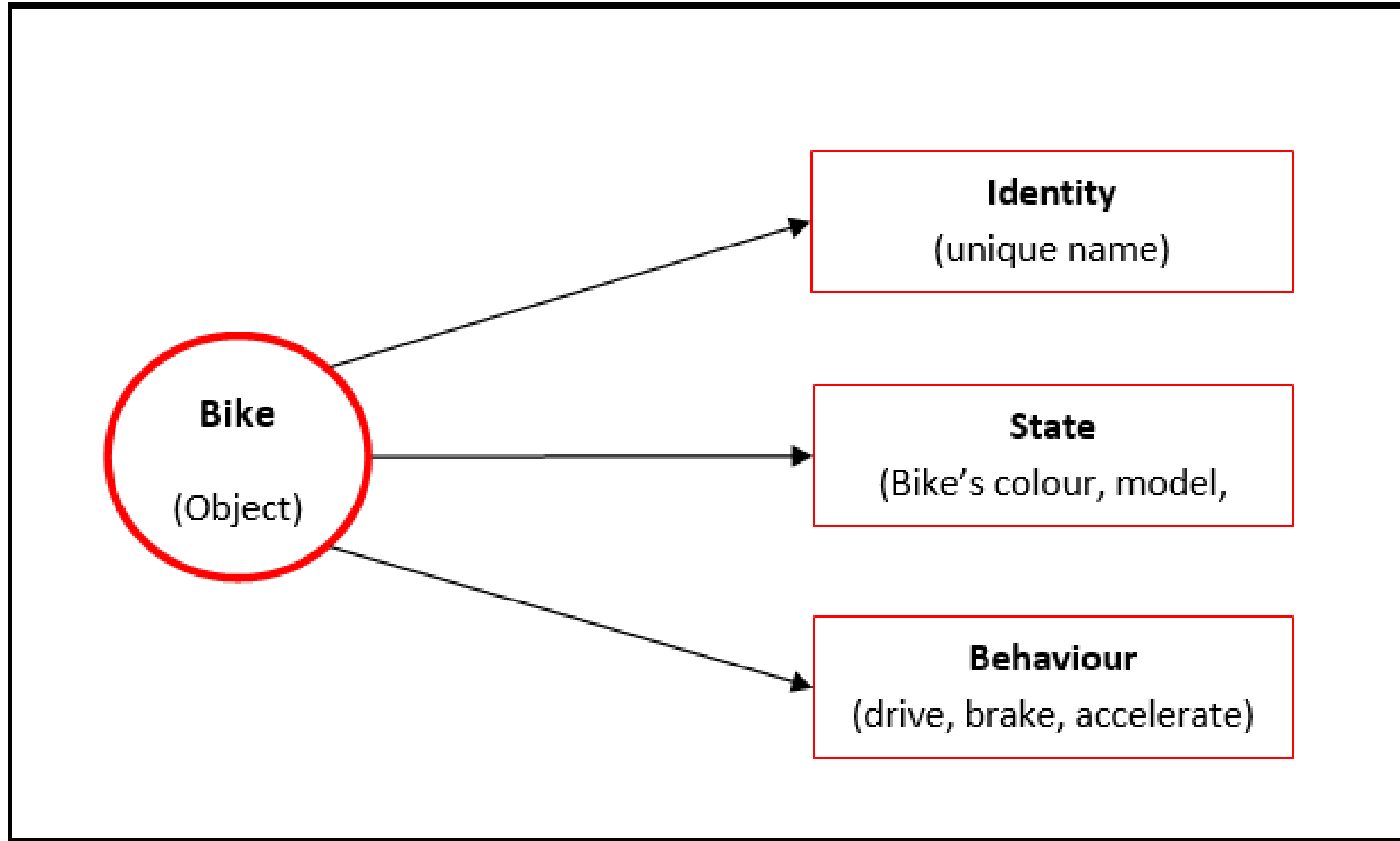
Behavior

represents the behavior of an object such as deposit, withdraw, etc.

C

Identity

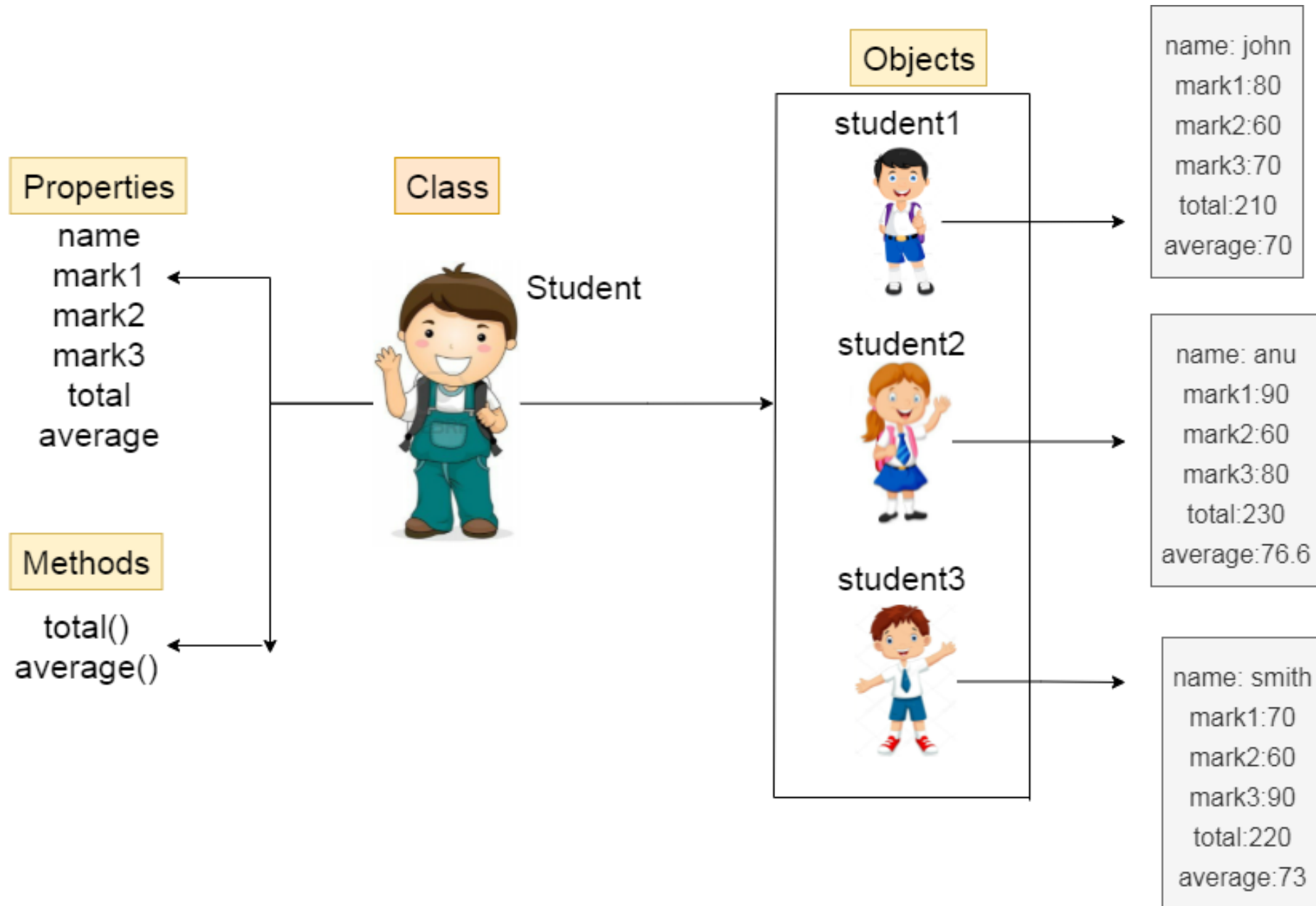
It is used internally by the JVM to identify each object uniquely.



Class

A class is a collection of objects and serves as a logical entity. It forms the basis for object-oriented programming in C++. To access and use the data members and member functions of a user-defined data type, a class instance must be created. A class functions as a blueprint for its objects.

Consider the example of a car class. While different cars may have distinct names and brands, they all share common features such as having four wheels, a speed limit, and a certain mileage range. In this scenario, the car class represents these shared characteristics, such as the wheels, speed limits, and mileage.



Inheritance

Inheritance is the process of creating a new class (derived class) from an existing class (base class). The derived class inherits the properties and methods of the base class, allowing code reuse and reducing redundancy.

Inheritance occurs when an object acquires all the properties and behaviors of its parent object. This feature promotes code reusability and enables runtime polymorphism.

- Subclass: Also known as a derived class, a subclass is a class that inherits properties from another class.
- Superclass: A superclass, or base class, refers to the class from which a subclass inherits its properties.
- Reusability: Inheritance allows the creation of a new class that builds upon an existing class with some desired functionality. This way, the new class can reuse the fields and methods of the pre-existing class, reducing redundancy and promoting maintainability.

Polymorphism

Polymorphism is a concept where a single task can be performed in different ways. For instance, convincing a customer using various methods or drawing various shapes like rectangles or circles. The behavior of an operation may vary depending on the situation or the type of data used.

Abstraction

Hiding internal details and showing functionality is known as abstraction. Data abstraction is the process of exposing to the outside world only the information that is absolutely necessary while concealing implementation or background information. For example: phone call, we don't know the internal processing.

Encapsulation

Encapsulation is the concept of bundling data and functions together within a class, hiding the internal workings of the class and improving security.

Example: A Car class encapsulates data like speed and color, along with methods like `accelerate()` and `brake()`.

Features of Java

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

Features of Java

1. Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand.

Features of Java

2. Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior..

Features of Java

2. Object-oriented

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

- ❖ Object
- ❖ Class
- ❖ Inheritance
- ❖ Polymorphism
- ❖ Abstraction
- ❖ Encapsulation

Features of Java

2. Object-oriented

What is an object in Java ?

An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical. The example of an intangible object is the banking system.

- ❑ **State:** represents the data (value) of an object.
- ❑ **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- ❑ **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

Features of Java

2. Object-oriented

What is a class in Java ?

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- ☐ **Fields**
- ☐ **Methods**
- ☐ **Constructors**
- ☐ **Blocks**
- ☐ **Nested class and interface**

Features of Java

2. Object-oriented

Syntax to declare a class:

```
class <class_name>{  
    field;  
    method;  
}
```

Features of Java

3. Platform Independent

- Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language.
- A platform is the hardware or software environment in which a program runs.
- There are two types of platforms software-based and hardware-based. Java provides a software-based platform.
- The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. It has two components:
 1. Runtime Environment.
 2. API(Application Programming Interface).

Features of Java

3. Platform Independent

- Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc.
- Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms

Features of Java

4. Portable

- Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

Features of Java

5. Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**

Features of Java

6. Robust

- The English meaning of Robust is strong. Java is robust because:
- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.

Features of Java

7. Architecture-neutral

- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture.
- However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Features of Java

7. Interpreted

Java can be considered both a compiled and an interpreted language because its source code is first compiled into a binary byte-code.

This byte-code runs on the Java Virtual Machine (JVM), which is usually a software-based interpreter.

Features of Java

9. High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.

It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Features of Java

10. Multi-threaded

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area.
- Threads are important for multi-media, Web applications, etc.

Features of Java

11. Distributed

- Java is distributed because it facilitates users to create distributed applications in Java.
- This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Features of Java

12. Dynamic

- Java is a dynamic language. It supports the dynamic loading of classes.
- It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

JVM

(Java Virtual Machine)

JVM

- ❑ JVM (Java Virtual Machine) is an abstract machine. It is called a virtual machine because it doesn't physically exist.
- ❑ It is a specification that provides a runtime environment in which Java bytecode can be executed.
- ❑ It can also run those programs which are written in other languages and compiled to Java bytecode.
- ❑ JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other.

However, Java is platform independent.

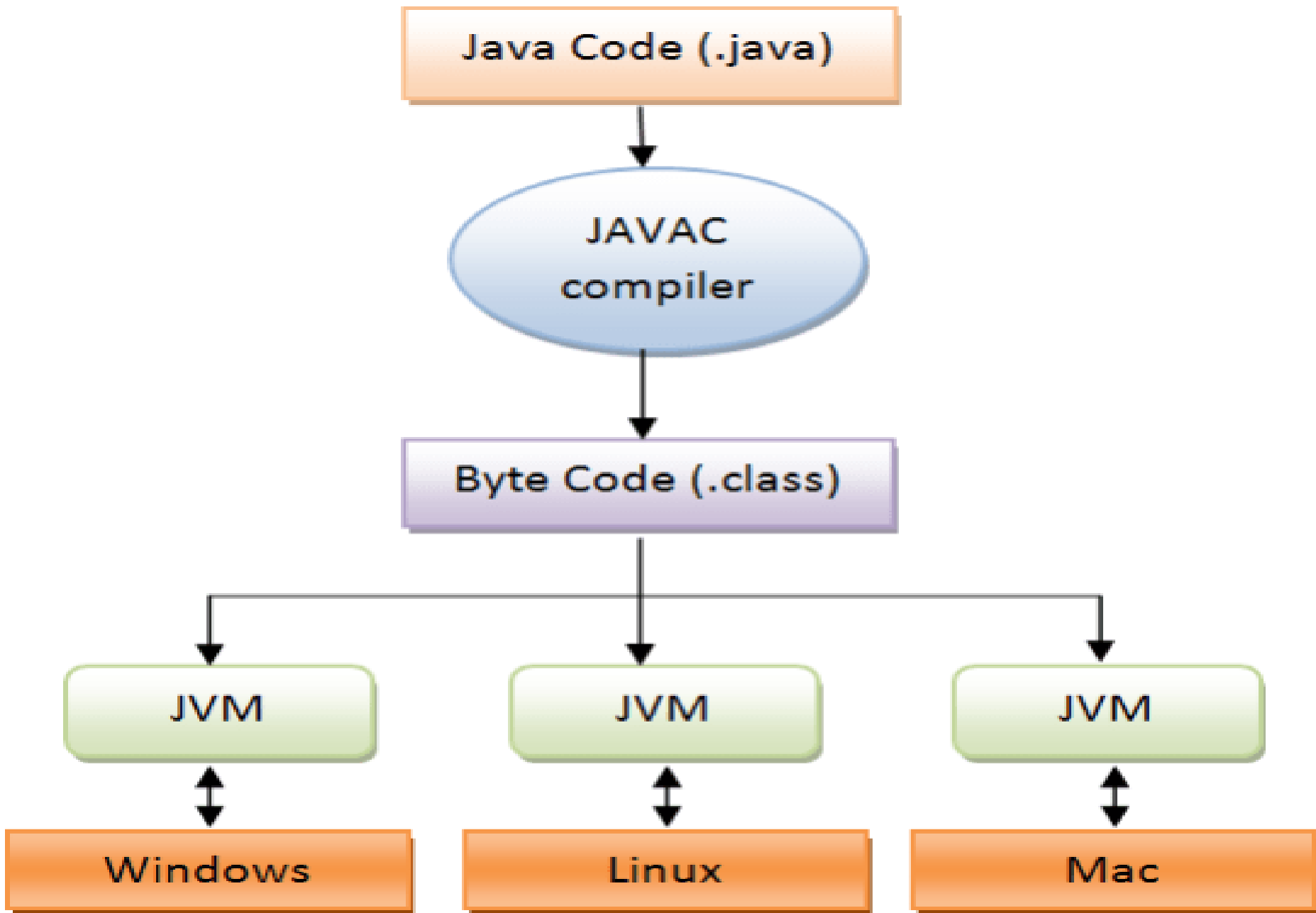
There are three notions of the JVM: specification, implementation, and instance.

The JVM performs the following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

Bytecode

- ❑ A bytecode in java is the set of instruction for JVM.
- ❑ Bytecode in Java is the reason java is platform-independent, as soon as a Java program is compiled bytecode is generated.
- ❑ To be more precise a Java bytecode is **the machine code in the form of a class file**.
- ❑ A bytecode in Java is the instruction set for Java Virtual Machine and acts similar to an assembler.



JRE

- JRE is an acronym for Java Runtime Environment.
- It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications.
- It is used to provide the runtime environment. It is the implementation of JVM.
- It physically exists.
- It contains a set of libraries + other files that JVM uses at runtime.

❑JDK

- ❑JDK is an acronym for Java Development Kit.
- ❑The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets.
- ❑It physically exists.
- ❑It contains JRE + development tools.
- ❑The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application